

Transcrição do Material interativo: Processos Leves

Página 1

Depois de estudarmos Processos Pesados, seguiremos com o estudo do *threads*.

Página 2

Threads são os diferentes fluxos de execução de um processo. Eles possuem informações próprias, como:

- um ID;
- um contador de programa;
- um conjunto de registradores;
- uma pilha.

Seção de Código, Seção de Dados e outros recursos do sistema operacional como arquivos abertos, por exemplo, são compartilhados. E por que os *threads* são chamados de Processos Leves? Justamente porque algumas seções são compartilhadas com outros *threads*, seções do próprio processo que criaram essas *threads*.

Página 3

Aqui temos um exemplo de um processo com um *thread* e outro com múltiplos *threads*. Vejam que ao lado esquerdo, há uma única *thread*. Neste caso, um programa que não implementa *threads* é a função *main*.

Percebam que ao lado direito, onde o processo possui várias *threads*, a seção de código, dados e arquivos é compartilhada entre elas, ou seja, apenas uma cópia dessas informações em memória.

O que é específico para cada *threads* são os registradores e a pilha, e esse é o correto, porque cada *threads* tem o seu próprio fluxo de execução. Eles podem executar a mesma função de um código, mas cada um pode estar executando um ponto diferente, e isso precisa ser levado em consideração.

Página 4

Alguns benefícios que podemos mencionar:

- Responsividade: posso ter um sistema, por exemplo, em que o lado servidor implementa diferentes *threads* para atender requisições de clientes.
- Compartilhamento de recursos: arquivos variáveis, ou seja, o que está no contexto do processo.
- Economia: em termos de uso dos recursos do sistema. O contexto individual é menor do que o contexto de um Processo Pesado, usa-se melhor a memória.
- Escalabilidade: onde as tarefas podem ser executadas em paralelo com as arquiteturas *multicore*.

Página 5

Quais são os desafios, então, da programação *multicore*?

Um deles é a identificação de tarefas, pois envolve análise das aplicações em busca de áreas que possam ser divididas em tarefas separadas e concorrentes. O ideal é que as

tarefas sejam independentes umas das outras e, portanto, possam ser executadas em paralelo aos núcleos individuais.

Outra questão é o equilíbrio. Além de identificar tarefas que possam ser executadas em paralelo, os programadores também devem assegurar que elas desenvolvam esforço igual e de mesmo valor. Em alguns casos, determinada tarefa pode não contribuir tanto para o processo em geral como outras. O uso de um núcleo de execução separado para a realização dessa tarefa pode não valer a pena. Isso significa que vai além da quantidade em termos de tarefas que são executadas; diz respeito ao esforço de processamento delas.

Outro ponto é a divisão dos dados. Da mesma forma que as aplicações são divididas em tarefas separadas, os dados acessados e manipulados por elas devem ser divididos para execução em núcleos separados. A dependência dos dados também é importante. Os dados acessados pelas tarefas devem ser examinados para determinar dependências entre duas ou mais tarefas. Quando uma tarefa é dependente dos dados de outra, os programadores devem assegurar que a execução delas seja sincronizada para acomodar a dependência de dados.

O teste de depuração também deve ser considerado. Quando um programa está sendo executado em paralelo, em múltiplos núcleos podem existir muitos caminhos de execução diferentes. O teste e a depuração de programas concorrentes é inerentemente mais difícil do que o teste e a depuração de aplicações em uma única *thread*.

Página 6

Aqui há um exemplo de execução concorrente e um sistema com um único núcleo ou com um único *core*. Podemos observar que todas as tarefas são executadas de forma sequencial. A tarefa 1 está executando; ou ela finalizou a sua execução; ou o seu *timer* foi encerrado. Então, a tarefa 2 é executada, depois a tarefa 3 e, na sequência, a tarefa 4. Vejam que a tarefa 1 voltou para execução. Isso significa que, naquele momento em que ela estava em execução, o *timer* dela foi finalizado e ela foi interrompida, por exemplo. Agora, ela segue o seu fluxo de execução, assim como as demais tarefas.

Página 7

Temos aqui um exemplo de execução paralela em sistemas *multicore*. Nesse exemplo, nós temos um processador com 2 núcleos. Então, o *core* 1 executa a tarefa 1 ao mesmo tempo em que o *core* 2 executa a tarefa 2. Assim que terminar o *timer* da tarefa 1 ou ela finalizar a sua execução, uma outra tarefa será trazida para a execução no *core* 1, nesse caso, a tarefa 3. O mesmo ocorre no *core* 2 que, após a tarefa 2, executa a tarefa 4. Observem que a execução de tarefas ocorre de forma paralela entre os *cores* e de forma concorrente dentro dos *cores*.

Página 8

As *threads* podem ser implementadas em nível de usuário por meio de bibliotecas como:

- POSIX *Threads(pthread)*.
- Win32/Win64 *threads*.
- Java *threads*.

Página 9

Para *threads* em nível de Kernel, significa que o próprio Kernel do sistema operacional deve dar suporte. Hoje, praticamente todos os sistemas operacionais têm suporte a *threads*.

Página 10

Nesse sentido, temos basicamente três modelos para definir o relacionamento entre as *threads* de usuário e as *threads* do Kernel, que são:

- muitos-para-um.
- um-para-um.
- muitos-para-muitos.

Página 11

Muitos *threads* em nível de usuário são mapeados para um único *thread* no Kernel. Atualmente, esse modelo é pouco utilizado, pois não consegue tirar proveito dos múltiplos núcleos de processamento. Isso acontece porque muitas tragédias de nível de usuário são mapeadas para um *thread* de Kernel. O gerenciamento dos *threads* é eficiente, pois é feito pela biblioteca de *threads* no espaço do usuário. Mas, nesse caso, o processo inteiro será bloqueado se um *threads* fizer um chamado de sistema bloqueador. Além disso, já que apenas um *thread* por vez pode acessar o Kernel, muitos ficam incapazes de executar em paralelo em sistemas *multicore*.

Página 12

O modelo um-para-um fornece maior concorrência do que o modelo muitos-para-um ao permitir que outro *thread* seja executado quando um *thread* faz chamada de sistema bloqueador. Ele também permite que múltiplos *threads* sejam executados em paralelo a multiprocessadores. A única desvantagem desse modelo é que a criação de um *thread* de usuário requer uma criação de *thread* de Kernel correspondente. Já que o *over head* de criação de *thread* de Kernel pode sobrecarregar o desempenho de uma aplicação, a maioria das implementações desse modelo restringe o número de *threads* suportados pelo sistema.

Página 13

No modelo muitos-para-muitos, permite-se que muitos *threads* em nível de usuário sejam mapeados para muitos *threads* em nível de Kernel. O número de *threads* de Kernel pode ser específico para determinada aplicação ou máquina. Uma aplicação pode receber mais *threads* de Kernel em um ambiente multiprocessador do que em um ambiente uniprocessador. Se analisarmos o modelo muitos-para-um, que permite que o desenvolvedor crie quantos *threads* de usuário quiser, ele não resulta em concorrência real, pois não há mesma quantidade de *threads* de Kernel correspondente.

O modelo um-para-um permite maior concorrência, mas deve-se cuidar para não criar *threads* demais dentro de uma aplicação. O modelo muitos-para-muitos não apresenta essas deficiências. Os desenvolvedores podem criar quantos *threads* de usuários foram necessários, e os *threads* de Kernel correspondentes podem ser executados em paralelo em um multiprocessador. Além disso, quando uma *thread* executa uma chamada de sistema

bloqueador, o Kernel pode designar no *sketch* outra *thread* para execução. A questão aqui é como definir a relação muitos-para-muitos em termos de quantidade?

Página 14

Então, para programarmos *threads*, existem bibliotecas que fornecem uma API. A API que utilizaremos como exemplo nas aulas será a POSIX *threads*.

Página 15

POSIX *threads* ou *Pthreads* é uma API utilizada para a criação e a sincronização de *threads*. A implementação fica a cargo do programador. *Pthreads* é comum em sistemas operacionais UNIX Like.